Exercise 4 – Hotel Management System

UML: Use Case · UM6P

1. Actors of the system

An actor is an external entity (person or system) that interacts directly with the application. Based on the text, we distinguish primary actors, who initiate use cases, from a secondary actor, which is an external system solicited by the application.

Actor	Type	Role in the system
Hotel client	Primary	Makes a reservation via the web interface or the mobile app, and pays the deposit online.
Receptionist	Primary	Records phone reservations, verifies reservations, searches for rooms, performs check-in and check-out, generates and completes invoices.
Cashier	Primary	Records the client's consumptions (breakfast, lunch, dinner).
Housekeeping staff	Primary	Marks rooms as cleaned (status “clean awaiting”).
Floor manager	Primary	Inspects rooms and sets them to “available” or “unavailable”.
Maintenance staff	Primary	Validates repairs so the room becomes available again.
Client's bank server	Secondary	External system that verifies and validates the online CIB card payment.

Not actors: the billing module is an internal component of the system, and SSL is a non-functional (security) constraint, not an entity interacting with the system.

2. Actions that occur outside the system

Some activities described in the text are physical or are performed by external systems. They are not use cases of the application, even though they are often followed by an update recorded in the system.

Action outside the system	Related action inside the system
The client calls the hotel by phone	The receptionist records the reservation
Verification and validation of the payment by the bank	The client makes the online payment
The client physically arrives at / leaves the hotel	Check-in / check-out by the receptionist
Serving meals (breakfast, lunch, dinner)	The cashier records the consumption
The client hands over the payment at the end of the stay	The receptionist completes the invoice
Physical cleaning of the rooms	Housekeeping marks the room as cleaned
Physical inspection of the rooms	The floor manager records the inspection result
Physical repair (air conditioner, water leak…)	Maintenance validates the repair

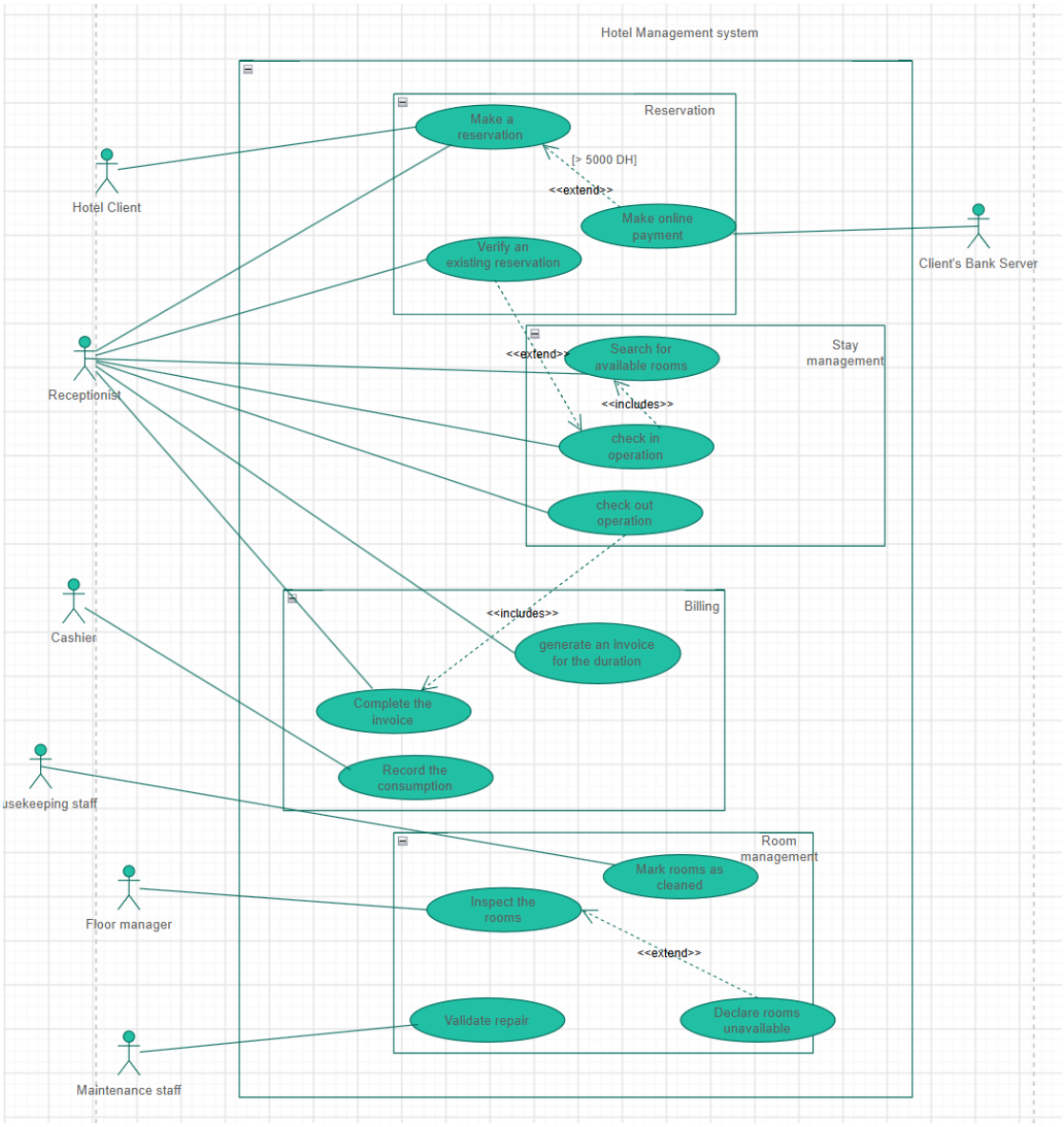
Automatic status changes (“occupied” at check-in, “dirty free” at check-out) and the automatic addition of charges by the billing module are effects of use cases, not use cases themselves.

3. Decomposition into boundaries (modules)

The system is decomposed by functional domain rather than by chronological step. Each module groups the use cases related to the same responsibility, and each actor mainly works within one module.

Module	Use cases	Main actors
Reservation	Make a reservation, Make online payment (if > 5000 DH), Verify an existing reservation	Client, Receptionist, Bank server
Stay management	Search for available rooms, Check-in, Check-out	Receptionist
Billing	Generate an invoice for the duration, Record the consumption, Complete the invoice	Receptionist, Cashier
Room management	Mark rooms as cleaned, Inspect the rooms, Declare rooms unavailable, Validate repair	Housekeeping staff, Floor manager, Maintenance staff

4. Use case diagram with functional boundaries



Justification of the relationships

Relationship	Justification
Make online payment <<extend>> Make a reservation	The payment is conditional: it only occurs when the estimated amount exceeds 5000 DH.
Verify an existing reservation <<extend>> Check-in	Verification only happens if the client already has a reservation.
Check-in <<include>> Search for available rooms	A suitable room must always be found before checking the client in.
Check-out <<include>> Complete the invoice	Every check-out closes the stay and completes the invoice after payment.
Declare rooms unavailable <<extend>> Inspect the rooms	A room is declared unavailable only if the inspection reveals a problem.